FITBUDDY

AI Fitness Planner

Complete Project Documentation — Explained Code, Architecture, Screenshots, Setup, Testing & References

Project type	Generative-AI powered FastAPI web application
Frontend	HTML + Jinja2 + CSS
Backend	FastAPI + Python
AI layer	Google Gemini Pro + Gemini Flash
Database	SQLite + SQLAlchemy
Persistence	Users, original plans, updated plans, feedback and tips

Prepared from the user-provided FitBuddy project documentation, the supplied source-code package, and the two attached application screenshots. The explanations below describe the supplied implementation rather than replacing it with a different architecture.

Contents

Section	Topic
1	Project Overview
2	Requirements and Functional Goals
3	System Architecture and Data Flow
4	Project Folder Structure
5	Backend Code — File-by-File Explanation
6	Frontend Code — File-by-File Explanation
7	AI / Gemini Integration
8	Database and Data Persistence
9	User Workflow and Screenshots
10	API Endpoints
11	Installation and VS Code Setup
12	Running and Testing the Application
13	Troubleshooting and Common Errors
14	Project Limitations and Safety Notes
15	References
Appendix	Complete Source Code

1. Project Overview

FitBuddy is described in the supplied project document as a web-based application that uses generative AI to create personalized 7-day workout plans, provide a practical nutrition or recovery tip, and revise an existing plan when the user submits feedback. The implementation combines FastAPI, Jinja2 templates, Google Gemini models, and SQLite/SQLAlchemy persistence. ■filecite■turn2file6■

The supplied source package implements the same overall workflow in a modular structure: routes coordinate the request, validation, AI calls and database operations; dedicated Gemini modules handle generation; repository functions isolate persistence; and Jinja2 templates render the user interface.

Core application flow

- User enters profile information on the home page.
- FastAPI receives and validates the submitted values.
- Gemini Pro is used for the structured workout plan.
- Gemini Flash is used for the concise nutrition/recovery tip.
- The user and generated plan are stored in SQLite through SQLAlchemy.
- The result page displays the plan and provides a feedback form.
- Feedback is sent with the existing plan to the update function, which creates a revised plan and stores it.
- An admin route provides a view of stored users and plans.

2. Requirements and Functional Goals

The original project documentation identifies the main AI requirements as structured 7-day workout generation, concise nutrition tips, feedback-based plan updates, and real-time web interaction. It also describes the modular FastAPI + Jinja2 + Gemini + SQLite architecture. ■filecite■turn2file0■

Requirement	How the supplied project addresses it
7-day plan	WorkoutPlan schema requires exactly seven days; Gemini workout generation returns structured JSON that is validated against the schema.
Nutrition / recovery tip	gemini_flash_generator.py calls the configured Flash model and validates the result as NutritionTip.
Feedback updates	updated_plan.py sends the existing plan plus user feedback to the configured Pro model and validates the revised plan.
Persistent storage	models.py defines User and Plan tables; repository.py contains create/update/read/delete operations.
Web interface	index.html collects input; result.html shows the generated plan; all_users.html provides the admin view.
API testing	main.py exposes FastAPI automatic documentation at /docs and routes.py provides JSON API endpoints.

3. System Architecture and Data Flow

The source document explicitly describes a modular architecture with four major layers: HTML/Jinja2 frontend, FastAPI backend, Gemini AI layer, and SQLite/SQLAlchemy database. ■filecite■turn1file0■

Layer	Files / components	Responsibility
Frontend	templates/*.html, static/style.css	Collect input, display generated plans, collect feedback, provide admin view.
Application / routing	app/main.py, app/routes.py	Create FastAPI app, mount static files, define web and JSON routes.
Validation / models	app/schemas.py, app/models.py	Validate incoming data and map persistent data to SQL tables.
AI layer	gemini_client.py, gemini_generator.py, gemini_flash_generator.py, updated_plan.py	Create Gemini client and generate or revise structured content.
Persistence	database.py, repository.py	Open DB sessions and implement user/plan CRUD operations.
Configuration	config.py, .env.example	Load environment variables and model/database settings.

Request flow: Browser → FastAPI route → Pydantic validation → Gemini generation → repository/database save → Jinja2 result page. For feedback, the flow becomes Browser → /submit-feedback → database lookup → updated_plan.py → Gemini → database update → result.html.

This separation is consistent with FastAPI's documented approach for larger applications, where related functionality is split into modules rather than keeping the whole application in one file. ■cite■turn0search7■

4. Project Folder Structure

```
FitBuddy_project_documentation/
■■■ app/
■   ■■■ __init__.py
■   ■■■ config.py
■   ■■■ database.py
■   ■■■ gemini_client.py
■   ■■■ gemini_flash_generator.py
■   ■■■ gemini_generator.py
■   ■■■ main.py
■   ■■■ models.py
■   ■■■ repository.py
■   ■■■ routes.py
■   ■■■ schemas.py
■   ■■■ updated_plan.py
■■■ static/
■   ■■■ style.css
■■■ templates/
■   ■■■ all_users.html
■   ■■■ error.html
■   ■■■ index.html
■   ■■■ result.html
■■■ tests/
■   ■■■ test_app.py
■■■ .env.example
■■■ .gitignore
■■■ README.md
■■■ requirements.txt
```

The app package contains Python modules. The templates folder contains Jinja2 HTML files, static contains CSS, and tests contains automated tests. This matches the source documentation's modular FastAPI/Jinja2 design. ■filecite■turn2file8■

5. Backend Code — File-by-File Explanation

app/config.py

Centralizes environment-based settings: database URL, Gemini key/model names, admin token, and feedback limit.

SOURCE CODE — excerpt from supplied project

```
app/config.py
from functools import lru_cache
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict
BASE_DIR = Path(__file__).resolve().parent.parent
class Settings(BaseSettings):
    app_name: str = "FitBuddy"
    app_env: str = "development"
    database_url: str = f"sqlite:///{BASE_DIR / 'fitbuddy.db'}"
    gemini_api_key: str = ""
    gemini_pro_model: str = "gemini-2.5-pro"
    gemini_flash_model: str = "gemini-3.8-flash"
    admin_token: str = "change-me"
    max_feedback_length: int = 2000
    model_config = SettingsConfigDict(
```

app/database.py

Creates the SQLite SQLAlchemy engine/session layer. get_db() yields a request session and closes it; init_db() creates missing tables.

SOURCE CODE — excerpt from supplied project

```
app/database.py
from collections.abc import Generator
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session
from .config import settings
connect_args = {"check_same_thread": False}
engine = create_engine(
    settings.database_url,
    connect_args=connect_args,
    future=True,
)
SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
```

5. Backend Code — AI and Validation

app/schemas.py

Defines Pydantic models for user input, feedback, seven-day workout output, exercises, and nutrition tips. WorkoutPlan is constrained to exactly seven days.

SOURCE CODE — excerpt from supplied project

```
app/schemas.py
from typing import Literal
from pydantic import BaseModel, Field, field_validator
Goal = Literal[
    "weight loss",
    "muscle gain",
    "general wellness",
    "flexibility",
    "endurance",
]
Intensity = Literal["low", "medium", "high"]
Experience = Literal["beginner", "intermediate", "advanced"]
class UserInput(BaseModel):
    user_id: str = Field(min_length=2, max_length=64)
    name: str = Field(min_length=2, max_length=100)
    age: int = Field(ge=13, le=100)
    weight: float = Field(gt=20, lt=400)
    goal: Goal
```

app/gemini_client.py

Creates the Gemini client and raises a clear configuration error when GEMINI_API_KEY is missing.

SOURCE CODE — excerpt from supplied project

```
app/gemini_client.py
from google import genai
from .config import settings
class GeminiNotConfigured(RuntimeError):
    pass
def get_client() -> genai.Client:
    if not settings.gemini_api_key:
        raise GeminiNotConfigured(
            "GEMINI_API_KEY is not configured. Add it to the .env file."
        )
    return genai.Client(api_key=settings.gemini_api_key)
```

5. Backend Code — Gemini Generation

app/gemini_generator.py

Builds the personalized seven-day workout prompt, requests structured JSON using WorkoutPlan, and validates the returned JSON before storage.

SOURCE CODE — excerpt from supplied project

```
app/gemini_generator.py
import json
from google.genai import types
from .config import settings
from .gemini_client import get_client
from .schemas import UserInput, WorkoutPlan
SYSTEM_INSTRUCTION = """
You are the workout-planning component of FitBuddy.
Create safe, practical general-wellness fitness guidance.
Do not diagnose medical conditions, prescribe treatment, recommend starvation,
extreme calorie restriction, dangerous exercises, or extreme training.
Do not promise body or health outcomes.
Respect the user's stated experience and intensity.
If pain, injury, or a medical condition is mentioned, recommend appropriate
professional guidance rather than diagnosing the user.
Return only data matching the supplied structured response schema.
"""
def generate_workout_gemini(user: UserInput) -> str:
```

app/gemini_flash_generator.py

Generates a concise nutrition/recovery tip with the configured Flash model and validates it as NutritionTip.

SOURCE CODE — excerpt from supplied project

```
app/gemini_flash_generator.py
import json
from google.genai import types
from .config import settings
from .gemini_client import get_client
from .schemas import NutritionTip
def generate_nutrition_tip_with_flash(goal: str, age: int) -> str:
    client = get_client()
    prompt = f"""
Create one concise, practical nutrition or recovery tip for a FitBuddy user.
Goal: {goal}
Age: {age}
Keep it focused on balanced eating, hydration, sleep, recovery, and sustainable
habits. Do not recommend crash diets, starvation, calorie restriction targets,
supplements, or medical treatment.
```

5. Backend Code — Persistence and Routes

app/models.py

Defines User and Plan SQLAlchemy tables and their relationship. Plans preserve the original plan, updated plan, feedback, and timestamps.

SOURCE CODE — excerpt from supplied project

```
app/models.py
from datetime import datetime, timezone
from sqlalchemy import DateTime, Float, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .database import Base
def utc_now() -> datetime:
    return datetime.now(timezone.utc)
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    name: Mapped[str] = mapped_column(String(100))
    age: Mapped[int] = mapped_column(Integer)
    weight: Mapped[float] = mapped_column(Float)
    goal: Mapped[str] = mapped_column(String(50))
```

app/repository.py

Keeps database CRUD operations separate from route logic, including user lookup, plan save/update, and deletion.

SOURCE CODE — excerpt from supplied project

```
app/repository.py
from sqlalchemy import select
from sqlalchemy.orm import Session
from .models import Plan, User, utc_now
from .schemas import UserInput
def get_user(db: Session, user_id: str) -> User | None:
    return db.scalar(select(User).where(User.user_id == user_id))
def save_user(db: Session, data: UserInput) -> User:
    user = get_user(db, data.user_id)
    if user is None:
        user = User(user_id=data.user_id)
        db.add(user)
    user.name = data.name
    user.age = data.age
    user.weight = data.weight
```

5–6. Backend Integration and Frontend

app/routes.py

Connects forms, validation, Gemini generation, repository operations, feedback updates, API endpoints, and error rendering.

SOURCE CODE — excerpt from supplied project

```
app/routes.py
import json
from pathlib import Path
from fastapi import APIRouter, Depends, Form, Request
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.templating import Jinja2Templates
from pydantic import ValidationError
from sqlalchemy.orm import Session
from .config import settings
from .database import get_db
from .gemini_client import GeminiNotConfigured
from .gemini_flash_generator import generate_nutrition_tip_with_flash
from .gemini_generator import generate_workout_gemini
from .repository import (
    delete_user,
    get_all_users,
    get_latest_plan,
    get_user,
```

app/main.py

Creates the FastAPI app, initializes the database at startup, mounts static files, and includes the router.

SOURCE CODE — excerpt from supplied project

```
app/main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from .config import settings
from .database import init_db
from .routes import router
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    yield
app = FastAPI(
    title=settings.app_name,
    description="Generative AI-powered fitness planning application.",
    version="1.0.0",
```

Frontend files

index.html collects the fitness profile; result.html displays the seven-day plan and feedback form; all_users.html provides the demo admin view; style.css supplies the visual design.

7. AI / Gemini Integration

The original project document says the initial model-selection work considered several Hugging Face models and ultimately selected Gemini Pro for structured workout generation and plan updating, with Gemini Flash for fast nutrition tips. ■filecite■turn1file2■

Function	Module	Configured role	Output
Initial workout	gemini_generator.py	Gemini Pro	WorkoutPlan JSON: seven days, exercises, warm-up/cooldown/recovery.
Nutrition / recovery	gemini_flash_generator.py	Gemini Flash	NutritionTip JSON.
Feedback update	updated_plan.py	Gemini Pro	Revised WorkoutPlan JSON.

Why structured output matters

Instead of treating Gemini's response as unrestricted text, the supplied implementation asks for JSON matching Pydantic schemas. That gives the application a predictable object structure before the response is stored or rendered. In particular, WorkoutPlan is constrained to seven days, which directly supports the project's seven-day requirement.

Environment configuration

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_PRO_MODEL=gemini-2.5-pro
GEMINI_FLASH_MODEL=gemini-3.8-flash
ADMIN_TOKEN=change-this-admin-token
```

Keep real API keys in .env and out of source control. The supplied .gitignore already excludes .env.

8. Database and Data Persistence

The project documentation states that user information and AI-generated plans are stored in SQLite through SQLAlchemy, while feedback-based revisions are stored separately so the original and updated versions can both be preserved. ■filecite■turn2file0■

Table	Important fields	Purpose
User	id, user_id, name, age, weight, goal, intensity, experience, created_at	Stores the user's profile and preferences.
Plan	id, user_id, original_plan, updated_plan, nutrition_tip, feedback, created_at, updated_at	Stores the generated plan and later revisions.

The local database file is generated from the configured SQLite URL. FastAPI's own database tutorial also notes that SQLite is convenient for a simple local application because it is file-based and supported by Python, while production deployments may use a database server. ■cite■turn0search2■

9. User Workflow and Attached Screenshots

The two supplied screenshots are included here as visual evidence of the running interface. They are not treated as source code; they illustrate the frontend/result states described in the project documentation.

9.1 Home / input interface

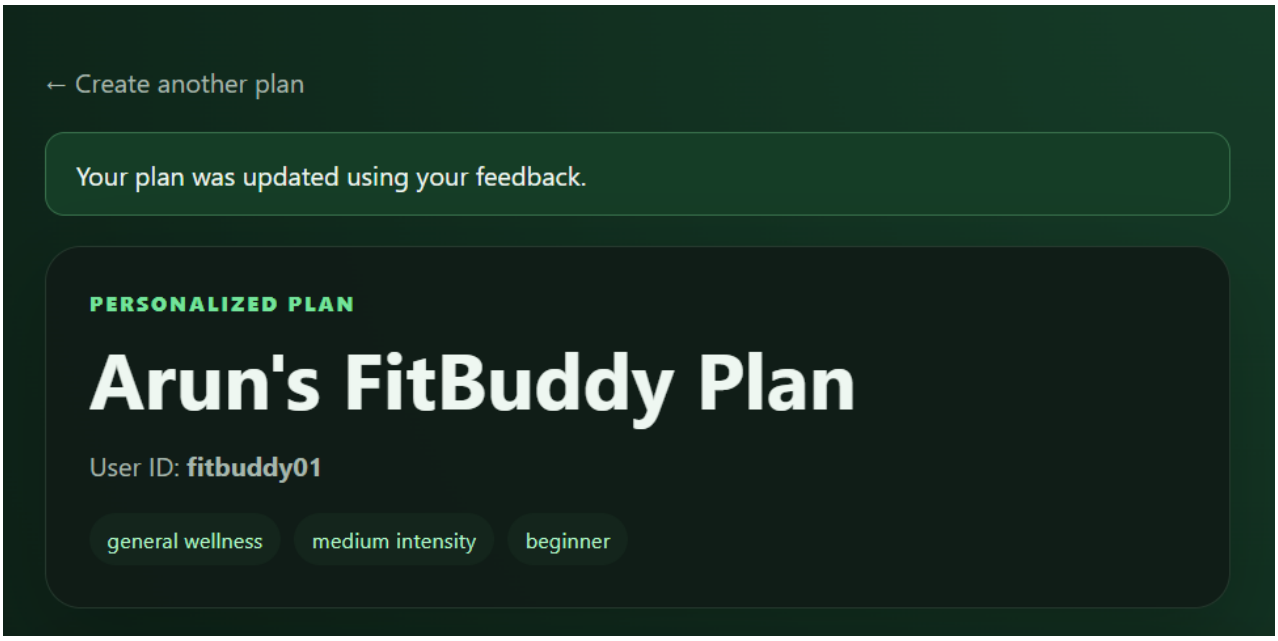


Figure 1. Supplied screenshot showing the FitBuddy home page and the Fitness Profile input section. The visible fields include User ID and Name, with the rest of the profile form continuing below the captured area.

9.2 Personalized result / updated plan interface

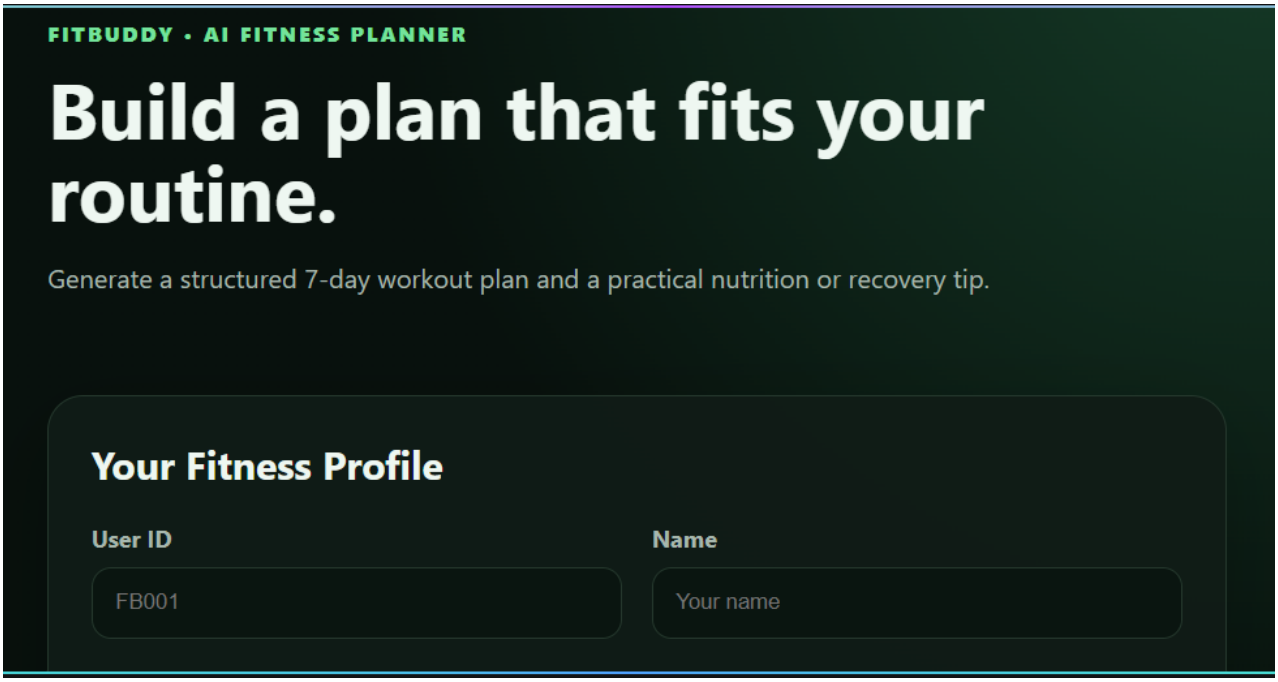


Figure 2. Supplied screenshot showing the personalized result state. The screenshot includes the confirmation message “Your plan was updated using your feedback.” and the FitBuddy plan header for the sample user.

Workflow represented by the screenshots

- Open the home page and enter a valid profile.
- Submit the form to generate the plan.
- Review the generated plan and nutrition/recovery tip.
- Submit feedback when a change is required.
- The updated plan is displayed with a confirmation message.

10. API Endpoints

Method	Path	Purpose
GET	/	Render the home page.
POST	/generate-workout	Validate form data, generate workout + tip, save them, and render result.
POST	/submit-feedback	Load latest plan, revise it using feedback, save update, render result.
GET	/view-all-users	Show admin user/plan dashboard after token check.
POST	/admin/users/{user_id}/delete	Delete a user when the demo admin token is valid.
GET	/api/health	Simple health response.
POST	/api/generate-workout	JSON API version of plan generation.
POST	/api/submit-feedback	JSON API version of feedback-based update.

FastAPI automatically exposes interactive API documentation at /docs when the application is running.
■cite■turn0search6■

11. Installation and VS Code Setup

The supplied README and project document use a Python virtual environment, dependency installation from requirements.txt, a .env file for the Gemini key, and Uvicorn for local execution. ■filecite■turn2file1■

Step 1 — Open the project

Open the FitBuddy_project_documentation folder in VS Code.

Step 2 — Create a virtual environment

In the VS Code terminal, run:

Step 3 — Activate it on Windows

Run:

Step 4 — Install dependencies

Run:

Step 5 — Create .env

Copy .env.example to .env and enter the required environment values.

Step 6 — Start the server

Run Uvicorn with app.main:app.

Step 7 — Open the application

Visit http://127.0.0.1:8000.

Step 8 — Test the API

Visit http://127.0.0.1:8000/docs.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

FastAPI's current documentation also supports the manual virtual-environment + pip workflow and documents the /docs interactive API interface. ■cite■turn0search3■turn0search6■

12. Running and Testing the Application

12.1 Browser test

- Start the server with Uvicorn.
- Open the home page.
- Enter a sample profile such as User ID FB001, a name, age, weight, goal, intensity and experience.
- Submit Generate Plan.
- Confirm that a seven-day plan and a nutrition/recovery tip appear.
- Enter feedback and submit it.
- Confirm that the updated-plan confirmation appears.

12.2 Automated test

```
pytest
```

The supplied tests exercise the home route, health endpoint, validation behavior and admin-token behavior.

12.3 API test through /docs

Open /docs after starting the server. FastAPI's interactive documentation lets you inspect the available endpoints and submit request bodies for the JSON API routes. ■cite■turn0search6■